



More on Time Complexity

Three Examples with Nested Loops

- Determine the time complexity

More Examples

- Example 4

```
int n=100;  
for(int i=n; i>0; i--) {  
    printf("Hello World\n");  
}
```

$0 \rightarrow n$

$1 \rightarrow n$

4

1 2 3 4

i takes the values:

$n, n-1, n-2, \dots, 0$

$n+1$

It runs $n+1$ times

Time complexity = $O(n)$

More Examples

- Example 5



```
int n=100;  
for(int i=1; i<n; i=i+2) {  
    printf("Hello World\n");  
}
```

i takes the values:
1, 3, 5, ..., n+sth

It runs for half of the values from 1 to n

Time complexity = $n/2 = O(n)$

More Examples

- Example 6



```
int n=100;  
for(int i=1; i<n; i=i+20) {  
    printf("Hello World\n");  
}
```

i takes the values:
1, 21, 42, ..., n+sth

It runs for 1/20 th of the values from 1 to n
(which is 5 in this case)

Time complexity = $n/20 = O(n)$

More Examples

- Example 7

```
int n=100;  
for(int i=0; i<n; i++) {  
    for(int j=0; j<i; j++) {  
        printf("Hello World");  
    }  
}
```

The diagram illustrates the recursive nature of the nested loops. A blue arrow points from the condition `i < n` in the outer loop to the handwritten expression $n+1$. Another blue arrow points from the condition `j < i` in the inner loop to the handwritten expression $n(n+1)$. The $n(n+1)$ expression has a checkmark next to it, indicating a completed step or a final state.

More Examples

- Example 7

```
int n=100;
for(int i=0; i<n; i++) {
    for(int j=0; j<i; j++) {
        printf("Hello World");
    }
}
```

i	j	stmt
---	---	------

More Examples

- Example 7

```
int n=100;  
for(int i=0; i<n; i++) {  
    for(int j=0; j<i; j++) {  
        printf("Hello World")  
    }  
}
```

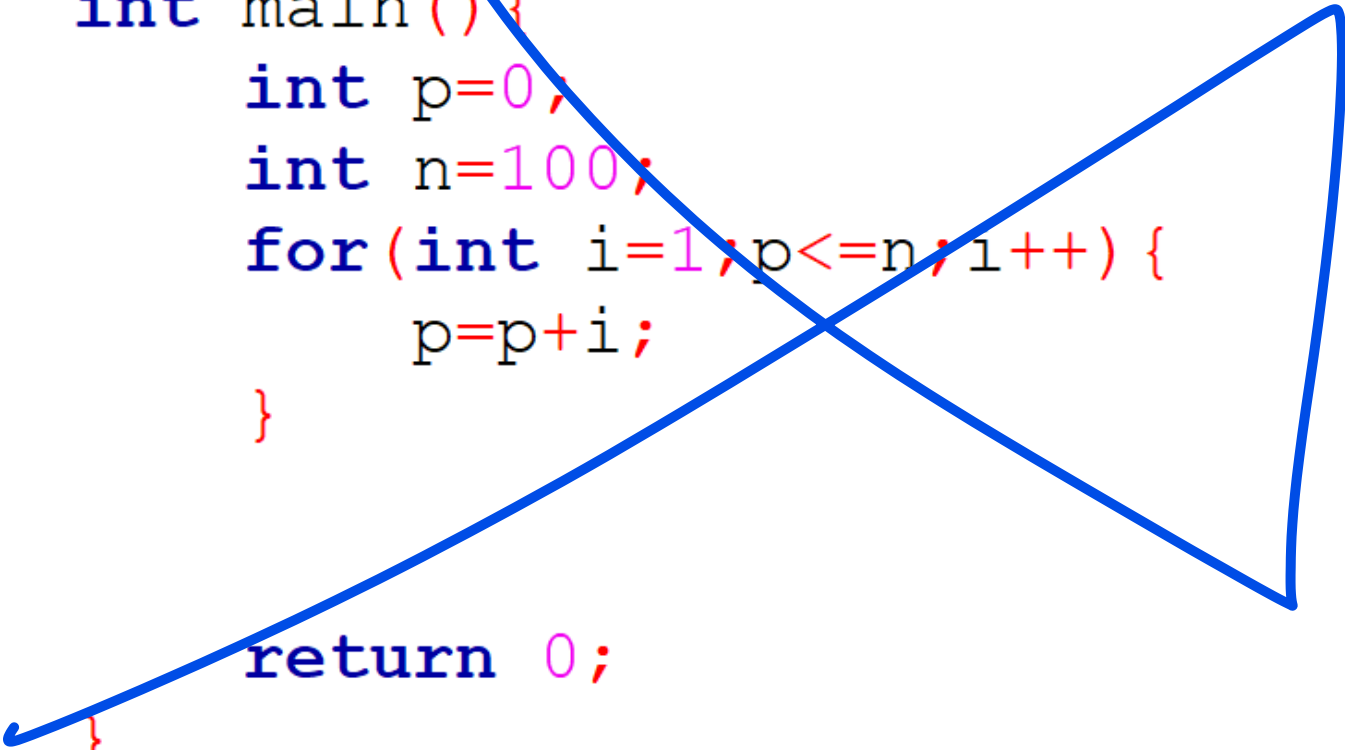
$$= \frac{n(n+1)}{2}$$

i	j	stmt
0	0	0
1	0	1
	1(X)	
2	0	2
	1	
	2(X)	
3	0	3
	1	
	2	
	3(x)	
...	...	...
n	0...n(x)	n

More Examples

- Example 8

```
#include<stdio.h>
int main () {
    int p=0;
    int n=100;
    for (int i=1; p<=n; i++) {
        p=p+i;
    }
    return 0;
}
```



i	p	n=100
1	0+1=1	
2	1+2=3	
3	3+3=6	
4	6+4=10	
k	1+2+3+...+k	

More Examples

- Example 8

```
#include<stdio.h>
int main() {
    int p=0;
    int n=100;
    for(int i=1; p<=n; i++) {
        p=p+i;
    }

    return 0;
}
```

i	p	n=100
1	0+1=1	
2	1+2=3	
3	3+3=6	
4	6+4=10	
k	1+2+3+...+k	

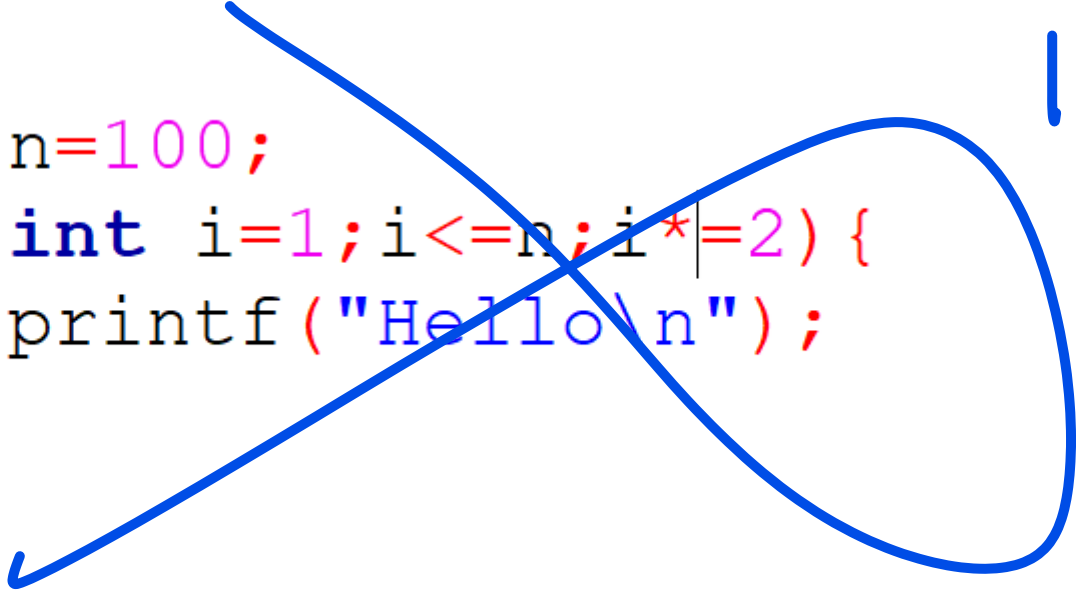
$p > n$
 $K(k+1)/2 > n$
 $O(\text{root}(n))$

Note: k is the number of times the loop ran

More Examples

- Example 9

```
int n=100;  
for(int i=1; i<=n; i*=2) {  
    printf("Hello\n");  
}
```



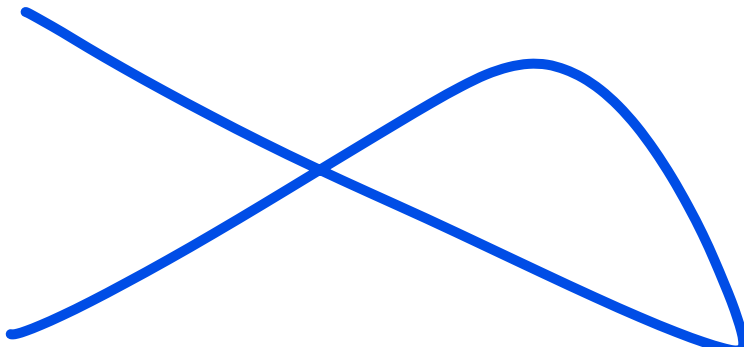
1 2 4 8 16 32

More Examples

- Example 9

How many times will the statement run?

```
int n=100;  
for(int i=1; i<=n; i*=2) {  
    printf("Hello\n");  
}
```



i	Printf()
1	1
2	1
4	1
...	...
2^k	1

More Examples

- Example 9

```
int n=100;  
for(int i=1; i<=n; i*=2) {  
    printf("Hello\n");  
}
```

How many times will the statement run?

i	Printf()
1	1
2	1
4	1
...	...
2^k	1

$2^k > n$
 $k > \log_2 n$
 $O(\log_2 n)$

Note: k is the number of times the loop ran

More Examples

- Example 10

```
int n=100;  
for(int i=n; i>=1; i=i/2) {  
    printf("Hello\n");  
}
```

$$n/2^k < 1$$

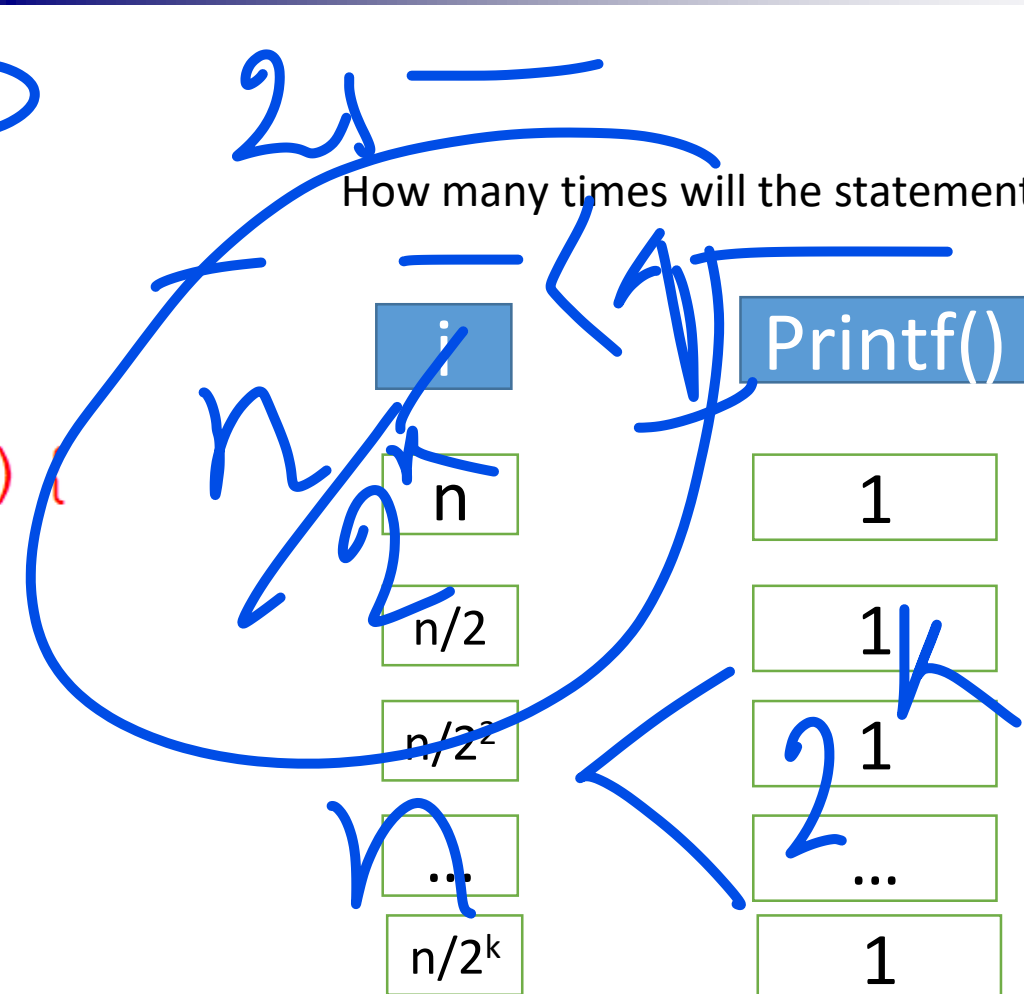
$$2^k > n$$

$$k > \log_2 n$$

$$O(\log_2 n)$$

Note: k is the number of times the loop ran

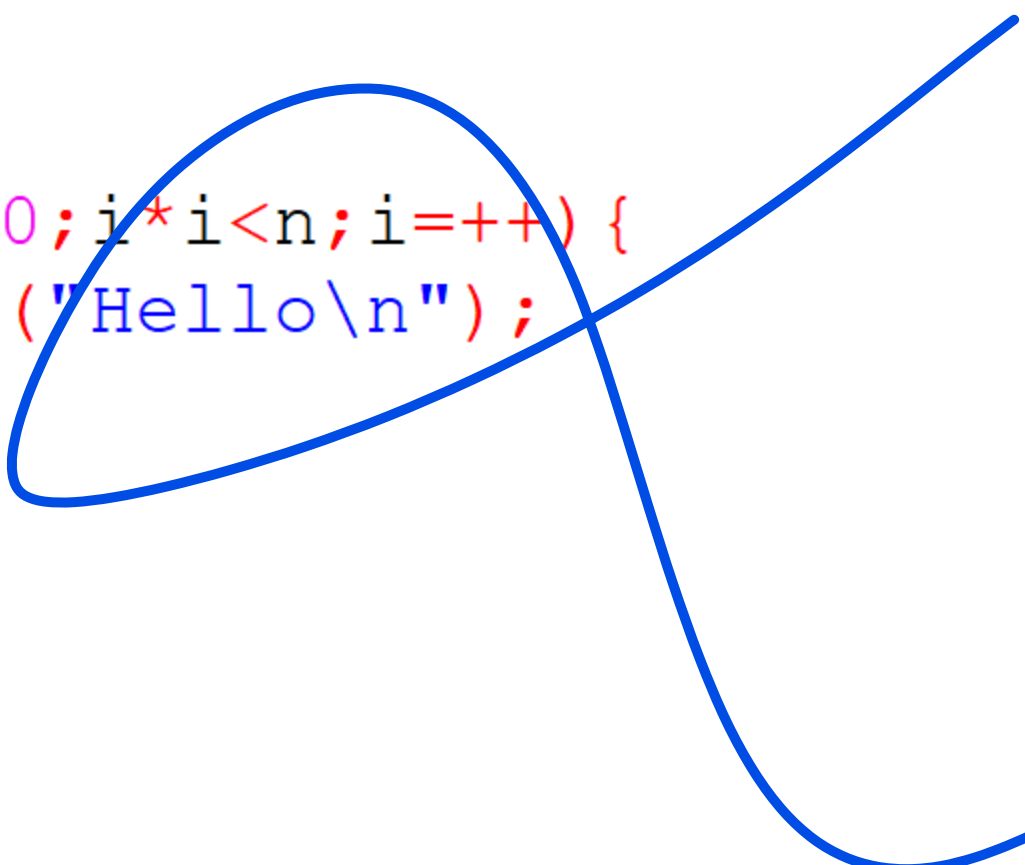
How many times will the statement run?



More Examples

- Example 11

```
int n=100;  
for(int i=0; i*i<n; i=++) {  
    printf("Hello\n");  
}
```



More Examples

- Example 11

0 1 2 3 —

*

```
int n=100;  
for(int i=0; i*i<n; i=++) {  
    printf("Hello\n");  
}
```

How many times will the loop run?

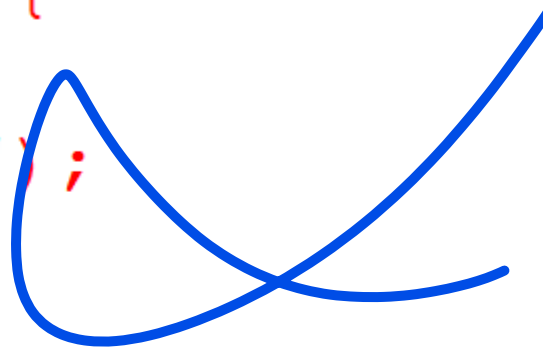
$i^2 < n$
 $i < \text{root}(n)$
 $O(\text{root}(n))$

$$i^2 < n \Rightarrow i < \sqrt{n}$$

More Examples

- Example 12

```
for (int i=0; i<n; i++) {  
    printf("Hello\n");  
}
```



```
for (int j=0; j<n; j++) {  
    printf("World\n");  
}
```

More Examples

- Example 12

```
for (int i=0; i<n; i++) {  
    printf("Hello\n");  
}
```

$O(n)$

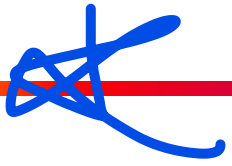
$O(2n) = O(n)$

```
for (int j=0; j<n; j++) {  
    printf("World\n");  
}
```

1×2

$2 \times 2 = 4$

4×2



More Examples

- Example 13 (determine the time complexity for the 2nd loop only)

```
int p=0;  
for(int i=1; i<n; i=i*2) {  
    p++;  
}
```

This loop will run $\log(n)$ times
Hence the value of p will be $\log(n)$

```
for(int j=1; j<p; j*j*2) {  
    printf("Hello");  
}
```

This loop will run $\log(p)$ times

Total time
= $O(\log p)$
= $O(\log \log(n))$
= $O(\log \log n)$

More Examples

- Example 14 (do it yourself)

```
int p=0;  
for(int i=0; i<n; i++) {  
    for(int j=0; j<n; j=j*2) {  
        printf("Hello world");  
    }  
}
```

$n+1$

$2^k > n$

$n \log_2 n$

More Examples

- Example 14 (do it yourself)

```
int p=0;
for(int i=0; i<n; i++) {
    for(int j=0; j<n; j=j*2) {
        printf("Hello world");
    }
}
```

$O(n \log n)$

More Examples

- Example 15

```
int i=0; ← 1

while (i<n) { ← n+1
    printf("Hello\n"); ← n
    i++; ← n
}
```

$O(n)$

More Examples

- Example 16

How many times will the statement run?

```
int i=1;  
int n=100;  
while (i<n) {  
    printf("Hello\n");  
    i=i*2;  
}
```

$2^k > n$

2^k

i	Printf()
1	1
2	1
4	1
...	...
2^k	1

More Examples

- Example 17

```
int main() {
```

```
int k=1;
```

```
int i=1;
```

```
int n=100;
```

```
while (k<n) {
```

```
    printf("Hello");
```

```
    k=k+i;
```

```
    i++
```

```
}
```

Handwritten diagram illustrating the execution of the code:

	k	i
1 →	2	2
2 →	4	3
	7	4
	11	5
	16	6

More Examples

Example 17

```
int main() {  
    int k=1;  
    int i=1;  
    int n=100;  
    while (k<n) {  
        printf("Hello");  
        k=k+i;  
        i++;  
    }
```

Note:

- i is increasing k
- k is used to terminate the loop

i	k	n=100
1	0+1=1	→ 2
2	1+2=3	→ 4
3	3+3=6	→ 7
4	6+4=10	
m	1+2+3+...+m	

More Examples

- Example 17

```
int main() {  
  
    int k=1;  
    int i=1;  
    int n=100;  
    while (k<n) {  
        printf("Hello");  
        k=k+i;  
        i++;  
    }
```

Note:

- i is increasing k
- k is used to terminate the loop

i	k	n=100
1	0+1=1	
2	1+2=3	
3	3+3=6	
4	6+4=10	
m	1+2+3+...+m	k>=n

More Examples

- Example 17

```
int main() {
```

```
int k=1;
```

```
int i=1;
```

```
int n=100;
```

```
while (k<n) {
```

```
    printf("Hello");
```

```
    k=k+i;
```

```
    i++
```

```
}
```

Note:

- i is increasing k
- k is used to terminate the loop

i	k	n=100
---	---	-------

1	0+1=1
---	-------

2	1+2=3
---	-------

3	3+3=6
---	-------

4	6+4=10
---	--------

m	1+2+3+...+m
---	-------------

$k \geq n$

$O(\sqrt{n})$

$$\frac{n(n+1)}{2}$$

$$\frac{n^2}{2}$$

$$\sqrt{n^2} = n$$



Thank You